14. Alpha-Beta Pruning Algorithm for Gaming:

Program:

```
def alpha_beta(depth, index, is_max, scores, alpha, beta, max_depth):
    if depth == max_depth:
        return scores[index]

    if is_max:
        best = float("-inf")
        for i in range(2):
            value = alpha_beta(
                depth + 1, index * 2 + i,
                False, scores, alpha, beta, max_depth
            )
            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break
        return best

    else:
        best = float("inf")
        for i in range(2):
            value = alpha_beta(
                depth + 1, index * 2 + i,
                True, scores, alpha, beta, max_depth
            )
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break
        return best


scores = [3, 5, 6, 9, 1, 2, 0, -1]
depth = 3

result = alpha_beta(
    0, 0, True, scores,
    float("-inf"), float("inf"), depth
)

print("Leaf scores:", scores)
print("Best score for MAX:", result)
```

Output:

```
Leaf scores: [3, 5, 6, 9, 1, 2, 0, -1]
Best score for MAX: 5
```

15. Decision Tree:

Program:

```
from sklearn.tree import DecisionTreeClassifier

# Input features: Outlook, Temperature
# 0 = Sunny/Cool, 1 = Rainy/Warm
X = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1],
    [0, 0],
    [1, 1]
]

# 0 = No, 1 = Yes
y = [0, 0, 1, 1, 0, 1]

model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0
)

model.fit(X, y)

prediction = model.predict([[1, 1]])[0]

print("Prediction (0 = No, 1 = Yes):", prediction)

if prediction == 1:
    print("Decision: Play")
else:
    print("Decision: Do not play")
```

Output:

```
Prediction (0 = No, 1 = Yes): 1
Decision: Play
```